# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 3 – CODING CHALLENGE

Solutions with Python Programs and Output Screenshots

| | | | |
|---|---|---|---|
| **Course Code:** | DSA03 | **Course Title:** | Natural Language Processing |
| **CO Assessed:** | CO1 – Imitation (L1) | **Weightage:** | 30% of CIA |
| **Student Name:** | Aryan.KS | **Reg. No.:** | 192325051 |
| **Section / Batch:** | | **Date:** | |

## Question 1 | Section 1: Intelligent Email and Password Validator using Regular Expressions

### Question

Problem Statement: A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

**Develop a Python application that validates Email Address, Password and Mobile Number using Regular Expressions.**

**Validation Rules**

**Email:**

- Must start with an alphabet.
- May contain letters, digits, '.', '_' before '@'.
- Domain should contain only alphabets.
- Valid extensions: .com, .org, .edu, .net, .in

**Password:**

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character (@, #, $, %, &, !)

**Mobile Number:**

- Must contain exactly 10 digits.
- First digit should be between 6–9.

**Expected Output: Valid Email / Invalid Email; Strong Password / Weak Password; Valid Mobile Number / Invalid Mobile Number**

### Answer *Python program: q1_email_password_validator.py*

```
"""Q1 - Intelligent Email, Password and Mobile Number Validator using Regular Expressions."""
import re

# Email: starts with a letter; letters/digits/'.'/'_' before '@';
#        domain only alphabets; extension in {com, org, edu, net, in}
EMAIL_RE = re.compile(r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(?:com|org|edu|net|in)$")

# Password: look-aheads enforce each rule independently; {8,} enforces minimum length
PASSWORD_RE = re.compile(r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$")

# Mobile: exactly 10 digits, first digit between 6 and 9
MOBILE_RE = re.compile(r"^[6-9]\d{9}$")

def validate_email(email):
    return "Valid Email" if EMAIL_RE.fullmatch(email) else "Invalid Email"
```

```
def validate password(pwd):
    return "Strong Password" if PASSWORD RE.fullmatch(pwd) else "Weak Password"

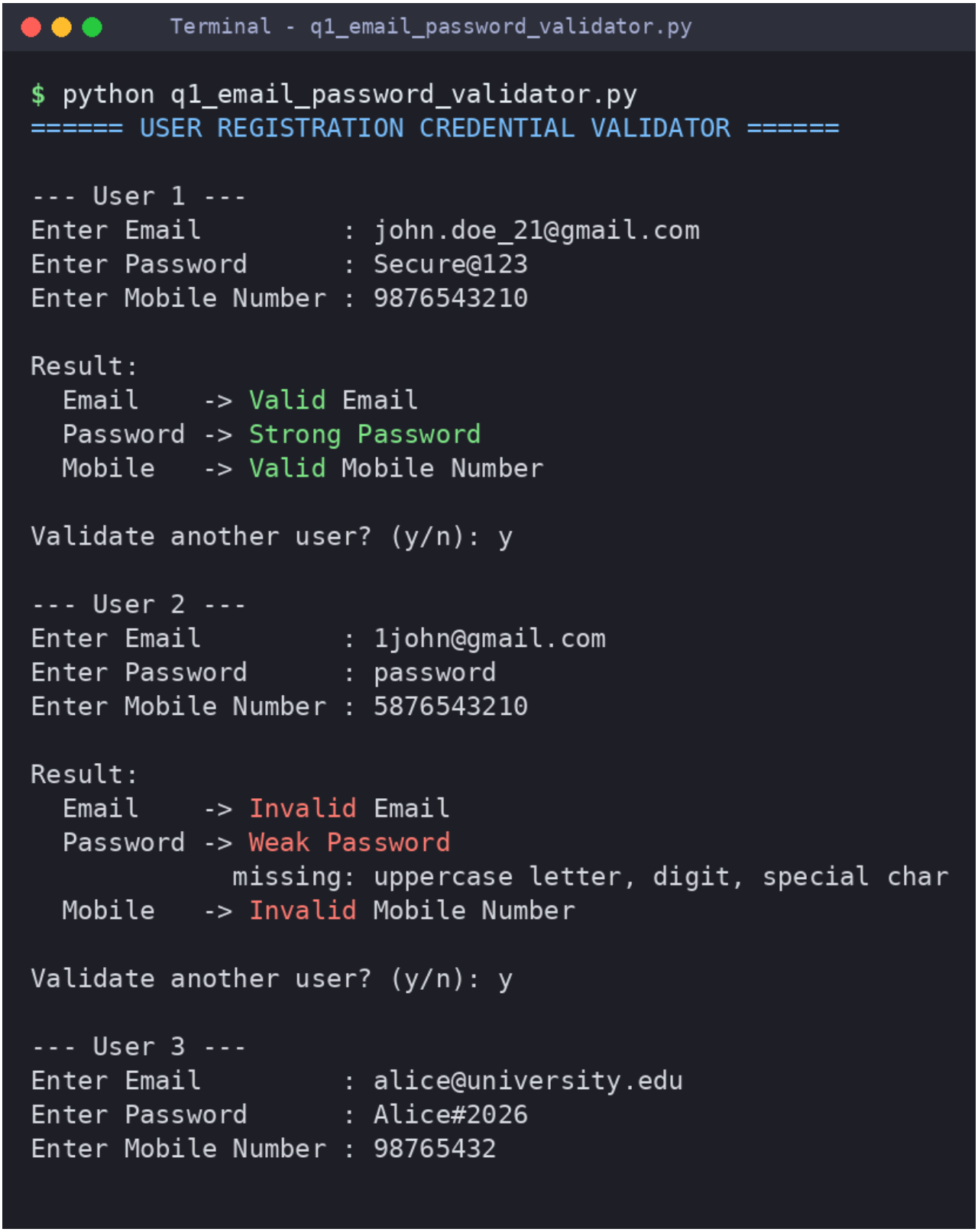
def validate mobile(mobile):
    return "Valid Mobile Number" if MOBILE RE.fullmatch(mobile) else "Invalid Mobile Number"

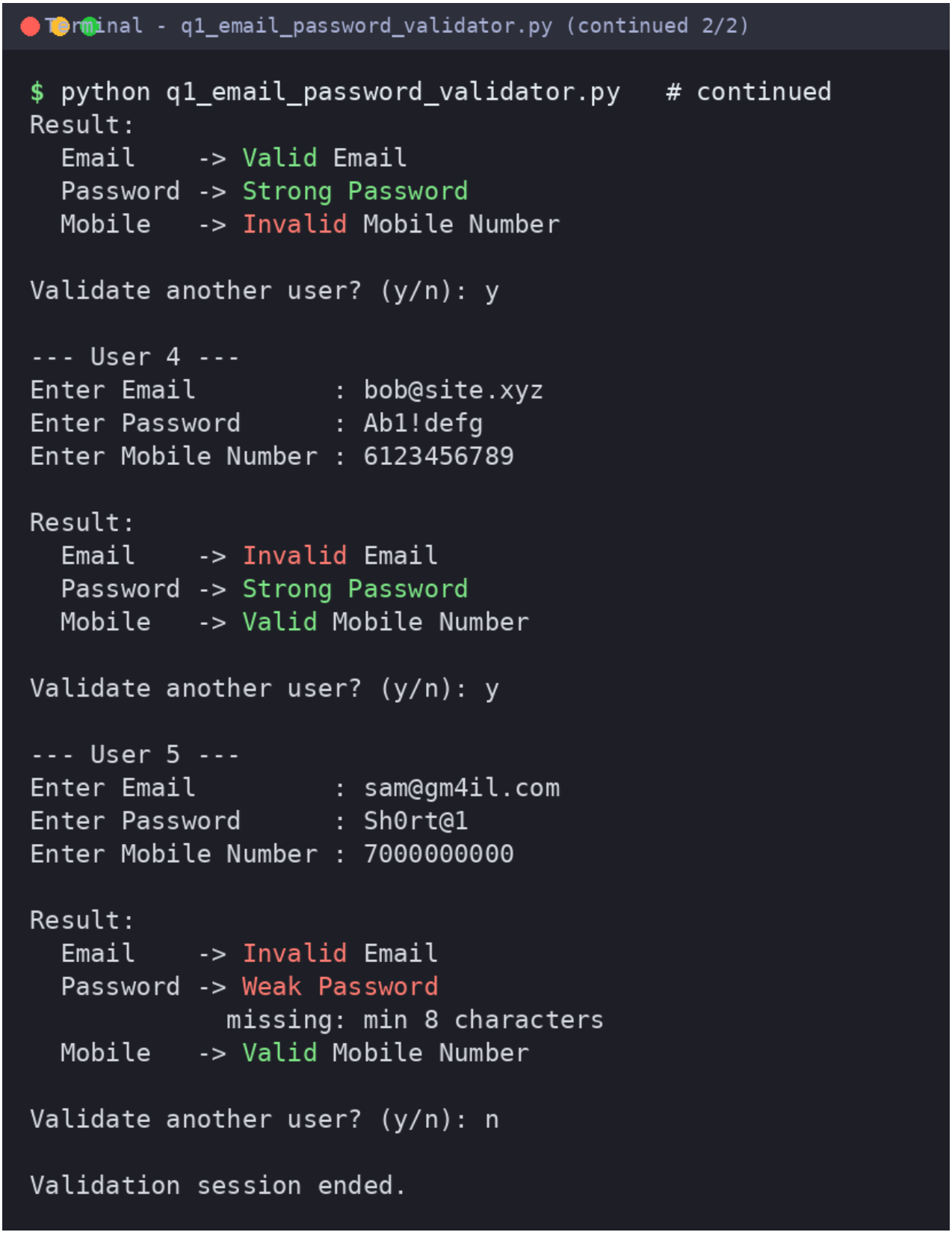
def password feedback(pwd):
    """List the policy rules that a weak password fails (helpful error handling)."""
    checks = [(r".{8,}", "min 8 characters"), (r"[A-Z]", "uppercase letter"),
              (r"[a-z]", "lowercase letter"), (r"\d", "digit"), (r"[@#$%&!]", "special char")]
    return [msg for rx, msg in checks if not re.search(rx, pwd)]

def main():
    print("====== USER REGISTRATION CREDENTIAL VALIDATOR ======")
    count = 0
    while True:
        count += 1
        print(f"\n--- User {count} ---")
        email = input("Enter Email         : ").strip()
        password = input("Enter Password      : ").strip()
        mobile = input("Enter Mobile Number : ").strip()
        print("\nResult:")
        print("  Email    ->", validate email(email))
        pw = validate password(password)
        print("  Password ->", pw)
        if pw == "Weak Password":
            print("             missing:", ", ".join(password feedback(password)))
        print("  Mobile   ->", validate mobile(mobile))
        if input("\nValidate another user? (y/n): ").strip().lower() != "y":
            break
    print("\nValidation session ended.")

if   name   == "  main  ":
    main()
```

**Output Screenshot**






**Justification of the output**

The e-mail regex enforces a leading letter, the allowed characters before “@”, an alphabetic-only domain and one of the five extensions. The password regex uses four look-aheads (upper-case, lower-case, digit, special character) plus .{8,} for length, so every rule is checked independently. The mobile regex needs exactly 10 digits starting with 6–9. User 1 passes everything; user 2 fails all three checks; users 3–5 show partial failures (8-digit mobile, “.xyz” extension, short password and a digit in the domain).

## Question 2 | Section 2: Design a Finite State Automata (FSA) Simulator

### Question

Problem Statement: Develop a Python-based Deterministic Finite Automata (DFA) simulator.

**The program should:**

- Accept the DFA description: states, input alphabet, transition table, initial state, final state(s).
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

**Example – Language: strings ending with "ab". Input: abaab Output: Transition Path and Accepted**

### Answer *Python program: q2_dfa_simulator.py*

```
"""Q2 - Deterministic Finite Automata (DFA) Simulator."""

def read_dfa():
    """Accept the complete DFA description from the user."""
    states = input("Enter states (space separated)        : ").split()
    alphabet = input("Enter input alphabet (space separated): ").split()
    initial = input("Enter initial state                   : ").strip()
    finals = set(input("Enter final state(s)                  : ").split())
    print("Enter transitions as: <state> <symbol> <next_state>   (type 'done' to finish)")
    delta = {}
    while True:
        line = input("  > ").strip()
        if line.lower() == "done":
            break
        src, sym, dst = line.split()
        delta[(src, sym)] = dst
    return states, alphabet, initial, finals, delta

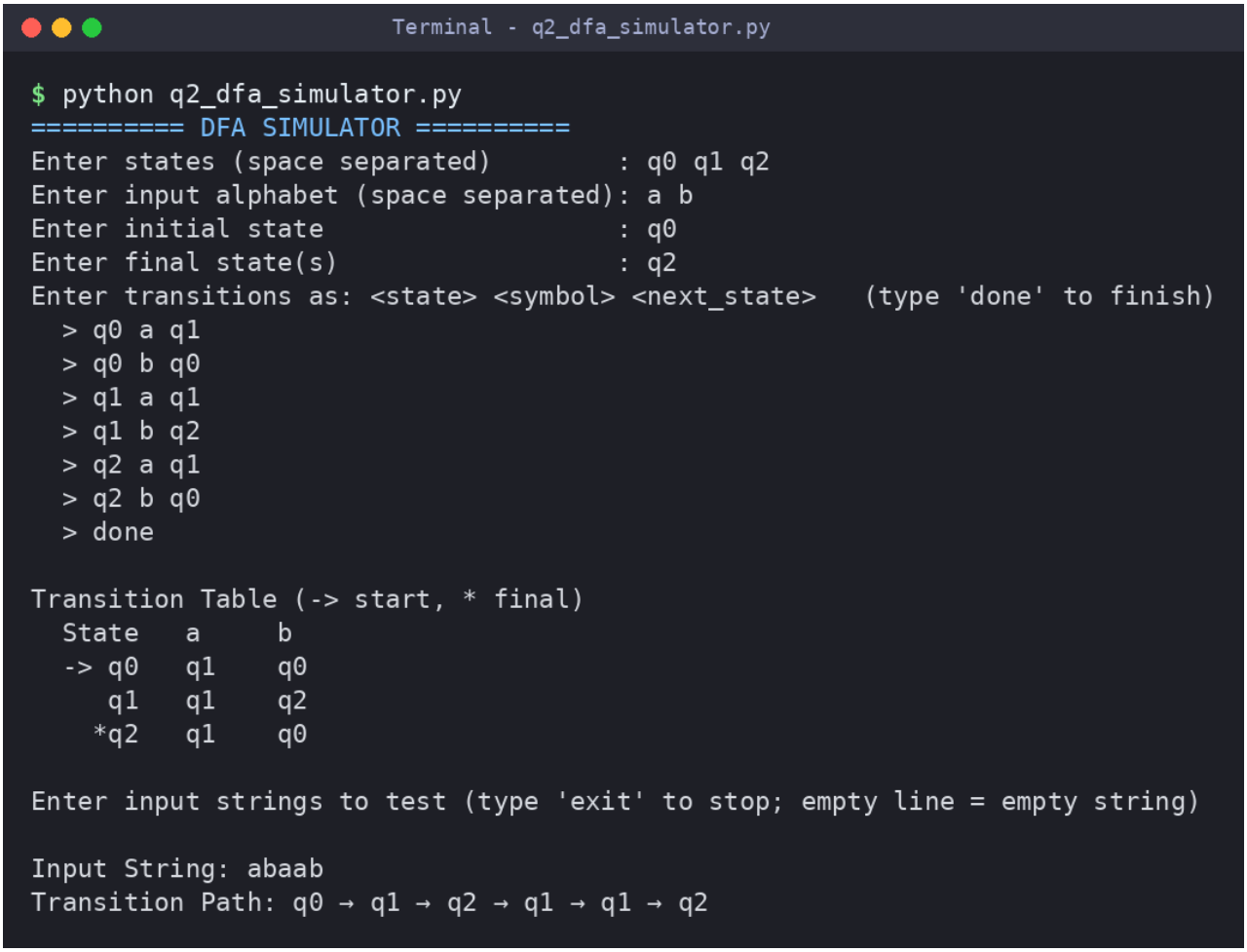
def print_table(states, alphabet, initial, finals, delta):
    print("\nTransition Table (-> start, * final)")
    print("  " + "State".ljust(8) + "".join(a.ljust(6) for a in alphabet))
    for s in states:
        mark = ("->" if s == initial else "  ")[:2]
        label = mark + ("*" if s in finals else " ") + s
        print("  " + label.ljust(8) + "".join(delta.get((s, a), "-").ljust(6) for a in alphabet))

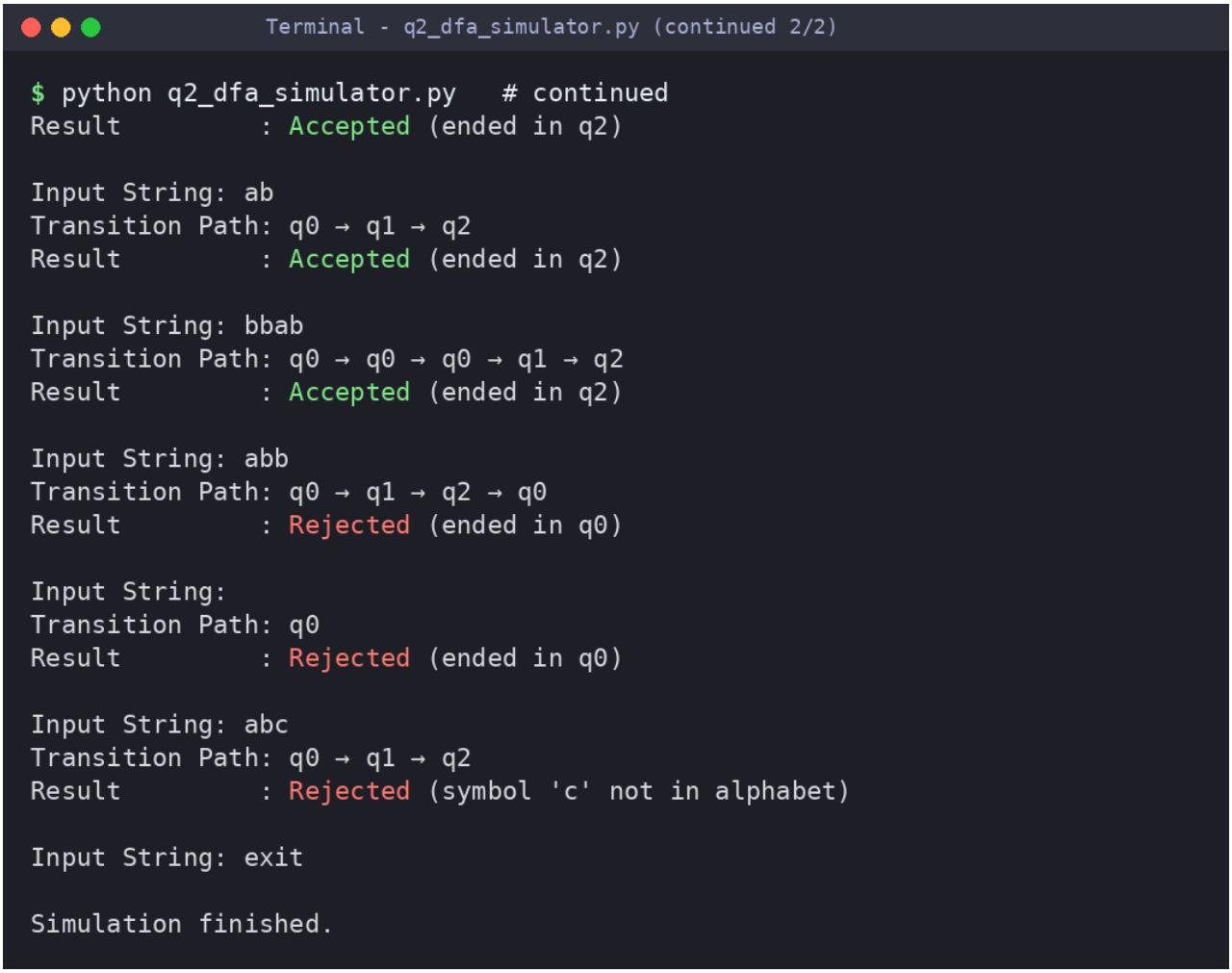
def simulate(string, initial, finals, alphabet, delta):
    """Run the DFA on a string; return (accepted, path, reason)."""
    state, path = initial, [initial]
    for ch in string:
        if ch not in alphabet:
            return False, path, f"symbol '{ch}' not in alphabet"
        if (state, ch) not in delta:
            return False, path, f"no transition from {state} on '{ch}'"
        state = delta[(state, ch)]
        path.append(state)
    return state in finals, path, f"ended in {state}"

def main():
    print("========== DFA SIMULATOR ==========")
    states, alphabet, initial, finals, delta = read_dfa()
    print_table(states, alphabet, initial, finals, delta)
    print("\nEnter input strings to test (type 'exit' to stop; empty line = empty string)")
    while True:
        s = input("\nInput String: ")
        if s.strip().lower() == "exit":
            break
        accepted, path, reason = simulate(s.strip(), initial, finals, alphabet, delta)
        print("Transition Path:", " → ".join(path))
        print("Result         :", "Accepted" if accepted else "Rejected", f"({reason})")
    print("\nSimulation finished.")

if __name__ == "__main__":
    main()
```

## Output Screenshot






### Justification of the output

The DFA is read from the user and stored as a dictionary δ[(state, symbol)] → next state. The simulator follows one transition per input symbol, records the path and accepts when the last state is a final state. For “strings ending with ab” (q0 –a→ q1 –b→ q2) the input abaab follows q0 → q1 → q2 → q1 → q1 → q2 and is Accepted. “abb” (ends in q0), the empty string and “abc” (symbol not in the alphabet) are Rejected.

## Question 3 | Section 3: Smart Pattern Matching Engine using Regular Expressions

### Question

Problem Statement: Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

**The system should support: Word Search, Prefix Search, Suffix Search, Date Search, Phone Number Search, Hashtag Search and Mention (@username) Search.**

Example Input: Meeting on 12/09/2026 | Call 9876543210 | #NLP | @OpenAI | natural language processing

User Menu: 1. Search Date 2. Search Phone Number 3. Search Hashtag 4. Search Mention 5. Search Prefix 6. Search Suffix

**Expected Output: Display all matching patterns based on user selection.**

### Answer *Python program: q3_pattern_matching_engine.py*

```
"""Q3 - Smart Pattern Matching Engine using Regular Expressions."""
import re

TEXT = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
Deadline: 25-12-2026, helpline +91-9123456780 #AI @simats edu"""

# Compiled patterns for the fixed search types
DATE_RE = re.compile(r"\b(?:0[1-9]|[12]\d|3[01])([/.-])(?:0[1-9]|1[0-2])\1\d{4}\b")
PHONE_RE = re.compile(r"(?<!\d)(?:\+91[-\s]?)?[6-9]\d{9}(?!\d)")
HASHTAG_RE = re.compile(r"#\w+")
MENTION_RE = re.compile(r"(?<![\w.])@\w+")

def find_all(regex, text):
    return [(m.start(), m.group()) for m in regex.finditer(text)]

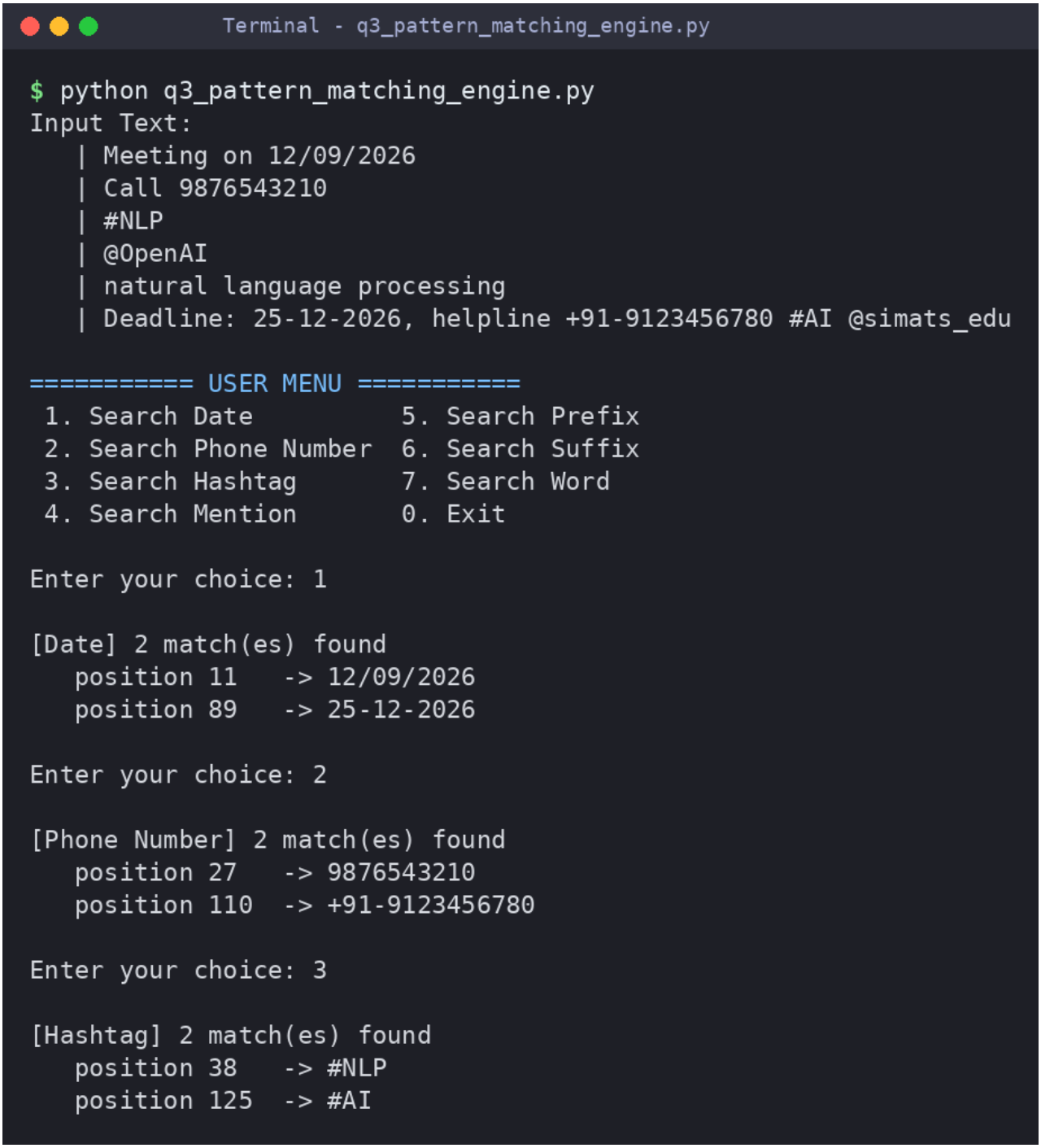
def word_pattern(word):   return re.compile(rf"\b{re.escape(word)}\b", re.IGNORECASE)
def prefix_pattern(pre):  return re.compile(rf"\b{re.escape(pre)}\w*", re.IGNORECASE)
def suffix_pattern(suf):  return re.compile(rf"\w*{re.escape(suf)}\b", re.IGNORECASE)

def show(title, matches):
    print(f"\n[{title}] {len(matches)} match(es) found")
    for pos, text in matches:
        print(f"   position {pos:<4} -> {text}")

def main():
    print("Input Text:")
    for line in TEXT.splitlines():
        print("   |", line)
    print("""
========== USER MENU ==========
 1. Search Date          5. Search Prefix
 2. Search Phone Number  6. Search Suffix
 3. Search Hashtag       7. Search Word
 4. Search Mention       0. Exit""")
    while True:
        choice = input("\nEnter your choice: ").strip()
        if choice == "1":   show("Date", find_all(DATE_RE, TEXT))
        elif choice == "2": show("Phone Number", find_all(PHONE_RE, TEXT))
        elif choice == "3": show("Hashtag", find_all(HASHTAG_RE, TEXT))
        elif choice == "4": show("Mention", find_all(MENTION_RE, TEXT))
        elif choice == "5":
            pre = input("Enter prefix: ").strip()
            show(f"Prefix '{pre}'", find_all(prefix_pattern(pre), TEXT))
        elif choice == "6":
            suf = input("Enter suffix: ").strip()
            show(f"Suffix '{suf}'", find_all(suffix_pattern(suf), TEXT))
        elif choice == "7":
            w = input("Enter word  : ").strip()
            show(f"Word '{w}'", find_all(word_pattern(w), TEXT))
        elif choice == "0":
            print("Exiting search engine. Goodbye!")
            break
        else:
            print("Invalid choice! Please select 0-7.")

if __name__ == "__main__":
    main()
```

### Output Screenshot



```
Terminal - q3_pattern_matching_engine.py (continued 2/2)

$ python q3_pattern_matching_engine.py   # continued

Enter your choice: 4

[Mention] 2 match(es) found
   position 43   -> @OpenAI
   position 129  -> @simats_edu

Enter your choice: 5
Enter prefix: nat

[Prefix 'nat'] 1 match(es) found
   position 51   -> natural

Enter your choice: 6
Enter suffix: ing

[Suffix 'ing'] 2 match(es) found
   position 0    -> Meeting
   position 68   -> processing

Enter your choice: 7
Enter word  : language

[Word 'language'] 1 match(es) found
   position 59   -> language

Enter your choice: 9
Invalid choice! Please select 0-7.

Enter your choice: 0
Exiting search engine. Goodbye!
```

**Justification of the output**

A menu-driven loop applies the chosen pattern with finditer and prints every match with its position. The date pattern uses a back-reference so the separator stays consistent (12/09/2026, 25-12-2026); phone, hashtag and mention patterns use look-arounds and \w+. Prefix “nat” returns “natural”; suffix “ing” returns “Meeting” and “processing”. An invalid menu choice prints an error message instead of crashing.